

2026 암호분석경진대회

3번 문제

1) 제공된 패킷 파일 `tls_live.pcap`을 분석하여 필요한 정보를 파악하고 자신의 급여가 500이 되도록 유효한 TLS application data를 생성(위조)하시오.

주어진 패킷을 분석하면 TCP 프로토콜의 3-way 핸드셰이크 과정과 TLS 1.2 핸드셰이크를 거쳐 암호화된 Application Data를 전송하는 과정임을 알 수 있다.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.2	10.0.0.1	TCP	40	54321 → 443 [SYN] Seq=0 Win=65535 Len=0
2	0.000000	10.0.0.1	10.0.0.2	TCP	40	443 → 54321 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
3	0.000000	10.0.0.2	10.0.0.1	TCP	40	54321 → 443 [ACK] Seq=1 Ack=1 Win=65535 Len=0
4	0.000000	10.0.0.2	10.0.0.1	TLSv1.2	176	Client Hello
5	0.000000	10.0.0.1	10.0.0.2	TLSv1.2	110	Server Hello
6	0.000000	10.0.0.1	10.0.0.2	TLSv1.2	836	Certificate
7	0.000000	10.0.0.1	10.0.0.2	TLSv1.2	345	Server Key Exchange
8	0.000000	10.0.0.1	10.0.0.2	TLSv1.2	49	Server Hello Done
9	0.000000	10.0.0.2	10.0.0.1	TLSv1.2	82	Client Key Exchange
10	0.000000	10.0.0.2	10.0.0.1	TLSv1.2	46	Change Cipher Spec
11	0.000000	10.0.0.2	10.0.0.1	TLSv1.2	85	Encrypted Handshake Message
12	0.000000	10.0.0.1	10.0.0.2	TLSv1.2	231	New Session Ticket
13	0.000000	10.0.0.1	10.0.0.2	TLSv1.2	46	Change Cipher Spec
14	0.000000	10.0.0.1	10.0.0.2	TLSv1.2	85	Encrypted Handshake Message
15	0.000000	10.0.0.1	10.0.0.2	TLSv1.2	103	Application Data
16	0.000000	10.0.0.1	10.0.0.2	TLSv1.2	119	Application Data
17	0.000000	10.0.0.1	10.0.0.2	TLSv1.2	117	Application Data
18	0.000000	10.0.0.1	10.0.0.2	TLSv1.2	109	Application Data
19	0.000000	10.0.0.2	10.0.0.1	TCP	40	54321 → 443 [ACK] Seq=230 Ack=1711 Win=65535 Len=0

그림 1. `tls_live.pcap`의 내용

TLS 1.2 프로토콜은 Server Hello(5번 패킷)을 통해 통신에서 사용할 암호화 방식을 결정한다. 해당 패킷을 조사하면 서버와 지사는 TLS_ECDHE_RSA_WITH_AES_GCM_SHA256 암호화 방식으로 통신함을 알 수 있다.

[Checksum Status: Unverified]	0000 45 00 00 6e 00 00 00 00 40 06 66 88 0a 00 00 01
Urgent Pointer: 0	0010 0a 00 00 02 01 bb d4 31 00 00 13 89 00 00 04 71
> [Timestamps]	0020 50 18 ff ff 52 ff 00 00 16 03 03 00 41 02 00 00
> [SEQ/ACK analysis]	0030 3d 03 03 55 d3 34 74 19 16 cf 36 12 f2 14 53 22
TCP payload (70 bytes)	0040 a6 af b4 4e bd 78 dd 16 d6 ef 62 9c e3 98 f2 5f
▼ Transport Layer Security	0050 f7 4c 0e 00 c0 2f 00 00 15 ff 01 00 01 00 00 0b
▼ TLSv1.2 Record Layer: Handshake Protocol: Server Hello	0060 00 04 03 00 01 02 00 23 00 00 00 17 00 00
Content Type: Handshake (22)	
Version: TLS 1.2 (0x0303)	
Length: 65	
▼ Handshake Protocol: Server Hello	
Handshake Type: Server Hello (2)	
Length: 61	
Version: TLS 1.2 (0x0303)	
> Random: 55d334741916cf3612f2145322a6afb44ebd78dd16d6e1	
Session ID Length: 0	
Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f)	
Compression Method: null (0)	
Extensions Length: 21	
> Extension: renegotiation_info (len=1)	
> Extension: ec_point_formats (len=4)	
> Extension: session_ticket (len=0)	
> Extension: extended_master_secret (len=0)	
[JA3S Fullstring: 771,49199,65281-11-35-23]	
[JA3S: 1af33e1657631357c73119488045302c]	

그림 2. 5번 패킷 정보의 일부

TLS_ECDHE_RSA_WITH_AES_GCM_SHA256 방식에서는 평문 P 에 대하여 다음과 같은 과정으로 생성된 키 스트림 KS 와의 XOR 연산을 수행하여 암호화를 진행한다. 여기서 K 는 대칭키, $Nonce$ 는 일회용 난수, i 는 카운터 값, n 은 평문 P 의 블록 수, E 함수는 AES 블록 암호화 함수, $MSB_{len(P)}$ 함수는 P 의 길이만큼 상위 비트를 추출하는 함수이다. KS_1 은 암호화에 사용되지 않고 인증태그 T 계산에 사용된다.

$$KS_i = E(K, \text{Nonce} \parallel [i]_{32}), KS = KS_2 \parallel KS_3 \parallel \dots \parallel KS_{n+1}$$

$$C = P \oplus MSB_{len(P)}(KS)$$

암호화의 무결성을 확인하는 인증태그 T 는 아래와 같은 연산으로 정의된다. 이때 A 는 추가 인증 데이터, 0^r 와 0^n 는 각각 A 와 C 가 128비트 단위로 정렬되도록 하는 패딩, $GHASH$ 는 AES-GCM에서 사용하는 범용 해시 함수이다.

$$H = E(K, 0^{128})$$

$$T = KS_1 \oplus GHASH_H(A, C)$$

이를 통해 최종적으로 TLS 1.2 환경에서 Encrypted Application Data(EAD)는 아래와 같이 구성되고 생략된 $\text{Nonce}[0:4]$ 는 TLS 핸드셰이크 과정에서 고정된다.

$$EAD = \text{Nonce}[4:12] \parallel C \parallel T$$

이제 15,16,17번 패킷의 평문을 각각 P_1, P_2, P_3 라 하자, 문제에서 위조 대상이 되는 17번 패킷의 메시지가 “급여를 0100 설정”이라고 주어졌으므로 P_3 가 “action=set_salary&uid={4자리숫자}&amt=0100&month={6자리숫자}”이라고 할 수 있다. 변조된 패킷 P'_3 은 “action=set_salary&uid={4자리숫자}&amt=0500&month={6자리숫자}”이고 아래와 같은 수식으로 C'_3 을 유도할 수 있다.

$$P'_3 = P_3 \oplus (0^{256} \parallel [4]_8 \parallel 0^{120})$$

$$C'_3 = P'_3 \oplus MSB_{len(P)}(KS) = P_3 \oplus (0^{256} \parallel [4]_8 \parallel 0^{120}) \oplus MSB_{len(P)}(KS) = C_3 \oplus (0^{256} \parallel [4]_8 \parallel 0^{120})$$

한편 15,16,17번 패킷에서 Nonce 값(0x9c269a9f29810ab0)을 재사용하므로 KS_1 이 일정하다. 따라서 15,16,17번 패킷의 인증태그를 T_1, T_2, T_3 라 하면 서로를 XOR 연산하여 H 에 대한 방정식 3개를 구할 수 있다.

$$T_1 \oplus T_2 = KS_1 \oplus GHASH_H(A_1, C_1) \oplus KS_1 \oplus GHASH_H(A_2, C_2) = GHASH_H(A_1, C_1) \oplus GHASH_H(A_2, C_2)$$

$$f(H) = (T_1 \oplus T_2) \oplus (GHASH_H(A_1, C_1) \oplus GHASH_H(A_2, C_2)) = 0$$

$$f(H) = (T_1 \oplus T_3) \oplus (GHASH_H(A_1, C_1) \oplus GHASH_H(A_3, C_3)) = 0$$

$$f(H) = (T_2 \oplus T_3) \oplus (GHASH_H(A_2, C_2) \oplus GHASH_H(A_3, C_3)) = 0$$

```

--- tag XOR ---
T1 ^ T2 : 9ace8a4cb803cd3131187cf5f737dbfe
T1 ^ T3 : b38b345b0bc982a474048b8476a74aaa
T2 ^ T3 : 2945be17b3ca4f95451cf77181909154

--- GHASH 입력값 ---
pk1 (seq=1): 00000000000000011703030022000000...0000000000000068000000000000110
pk2 (seq=2): 00000000000000021703030032000000...0000000000000068000000000000190
pk3 (seq=3): 00000000000000031703030030000000...0000000000000068000000000000180

```

그림 3. 위 내용의 진행 과정

해당 방정식들은 $GF(2^{128})$ 상에 정의된 유한체 다항식 형태를 띠며, 알려진 알고리즘(Cantor-Zassenhaus, Berlekamp 등)을 활용하여 다항 시간 내에 해를 구할 수 있다. 3개 방정식의 공통된 근을 찾으면 H 를 구할 수 있다.

```

### [1] pk1-pk2 pair candidates ###
- 112332a84132bc0c5c23a61037723683
### [2] pk1-pk3 pair candidates ###
- 112332a84132bc0c5c23a61037723683
### [3] pk2-pk3 pair candidates ###
- 112332a84132bc0c5c23a61037723683
- a2d57fe5b0fb465c5ca9f439c3f7bf66

=====
[+] Final Common Candidate for H (1 found):
>> 112332a84132bc0c5c23a61037723683
=====

```

그림 4. 근을 찾는 과정

H 를 구했으므로 아래와 같은 식으로 KS_1 을 구할 수 있다.

$$KS_1 = T \oplus GHASH_H(A, C)$$

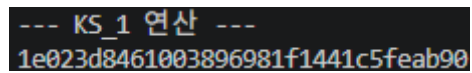


그림 5. KS_1 의 값

이를 활용하여 아래와 같은 수식으로 위조된 태그 T' 을 생성할 수 있고 최종적으로 문제에서 요구한 위조된 TLS record의 hex 문자열을 구할 수 있다.

$$T' = KS_1 \oplus GHASH_H(A, C')$$

17030300489c269a9f29810ab0a99141d1d84c3df6bfdcdb7dec7cf30462db203fbc36a29772edbf159643b468f1f1e52
0e7341daee867a49665542cd9f01f5a9c906b73ba3e13bbfea0e98a18